GOLDENHOUR / PROJECT BLUEPRINT

Evidence behind every proposal.

RAG, vector data and the implementation roadmap

A complete four-stage plan, a decision and credential questionnaire, and a clear explanation of the technology behind emergency hospital coordination.

01
FOUNDATION
Scope, data and shared contracts

02
KNOWLEDGE
Ingestion, publication and retrieval

03
AGENT 1
Grounded proposals and evidence review

04
DELIVERY
Evaluation, rollout and demonstration

The architectural decision
Use PostgreSQL for live operational facts and transactions. Add pgvector for approved reference knowledge. Agent 1 retrieves evidence and proposes requirements; clinicians confirm them.

Prepared 24 October 2026
Based on repository inspection and primary provider documentation.
This document proposes future work; it does not claim a validated clinical deployment.

Guide contents

Start with the project overview and architecture. Complete the input questionnaire before implementing the four stages. Use the final sections for technical handoff and demonstrations.

1. The project in plain language 3

2. What exists in the codebase today 4

3. Recommended technology stack and why it fits 5

4. Target architecture and responsibilities 7

5. Required user inputs and API questionnaire 8

6. Stage 1 - Confirm scope, persistence and shared contracts 12

7. Stage 2 - Build ingestion, publication and retrieval 14

8. Stage 3 - Integrate Agent 1 RAG and evidence review UI 17

9. Stage 4 - Evaluate, harden, deploy and prepare the demonstration 20

10. File-level implementation map 22

11. How to explain the project to anyone 24

12. References and verification notes 25

Reading guide

Product owners: sections 1, 4, 5 and 11.
Implementers: sections 2-10.
Reviewers: input ownership, acceptance gates and measured evaluation results.

1. The project in plain language

GoldenHour helps an ambulance crew and receiving hospitals coordinate an emergency transfer. It captures the patient's observations, proposes the kinds of hospital resources needed, checks which participating hospitals can support those requirements, compares time-to-care, and coordinates the destination and resource lifecycle.

The proposed upgrade gives Agent 1 an approved reference library. Before proposing requirements, Agent 1 retrieves relevant passages, uses them as evidence, and links its proposals to those passages. A clinician still reviews and confirms the requirements. Hospital eligibility, live resource counts, route times, and reservations remain separate services with explicit rules.

Thirty-second explanation: "GoldenHour connects ambulance crews with receiving hospitals. Its first agent uses approved reference documents to help explain proposed care requirements. A clinician confirms those requirements. The next stages check actual hospital resources and estimate time-to-care, then coordinate acceptance and equipment reservations. Maps show the journey; the audit trail explains what happened and why."

This is a synthetic-data demonstration until the separate operational, data-governance and clinical review work is completed. Adding retrieval or citations does not validate clinical correctness.

1.1 What the upgrade actually adds

- An approved, versioned knowledge library with document owners and review dates.
- Embeddings and vector search for finding passages by meaning.
- Keyword search alongside vector search for exact terms, acronyms and capability codes.
- A bounded RAG pipeline for Agent 1 with citations, structured output and explicit abstention.
- A source viewer in the assessment screen and a knowledge-management screen for authorized reviewers.
- Evaluation datasets, retrieval diagnostics, failure handling and reproducible audit metadata.

1.2 Important distinction: three kinds of data

Data class	Examples	Authoritative storage and access
Operational facts	Cases, hospital status, available beds, reservations, staff, clinician confirmation	Relational PostgreSQL tables, transactions, version checks
Reference knowledge	Approved protocols, capability definitions, local transfer procedures	Private document storage plus PostgreSQL metadata, chunks and pgvector embeddings
External observations	GPS fixes, Google road routes, traffic estimates	Explicit source adapters with timestamps and provider-specific handling

"Use a vector database for data" should mean vector indexing for reference knowledge, not replacing every table with embeddings. Similarity search cannot safely answer "are two ventilators available right now?" or atomically reserve the last bed.

2. What exists in the codebase today

This assessment reflects source inspection on the preparation date. It is not a claim that every feature is deployed, tested or connected to real services. Existing local changes were present before this documentation task and were left untouched.

Path or area	Observed responsibility	Implication for this plan
`apps/web`	React 18, TypeScript, Vite, Tailwind, React Router, Lucide	Reuse the web stack and existing role pages
`apps/api`	Fastify 5, TypeScript, Socket.IO, pg, Zod, Google GenAI SDK	Add retrieval and ingestion modules within the API architecture
`packages/contracts`	Shared Zod schemas and inferred TypeScript DTOs	Add evidence and retrieval contracts here
`packages/scenario-fixtures`	Seeded hospitals, roads and demonstration scenarios	Keep demonstration fixtures distinct from approved knowledge
`agent1.ts`	Deterministic proposal function plus asynchronous interpreter entry point	Preserve the entry point; introduce a RAG interpreter behind it
`interpreter.ts`	Gemini interpreter with deterministic fallback	Replace silent equivalence between fallback and grounded output with explicit modes
`gemini.ts`	Direct prompt-to-JSON generation; hardcoded model identifier	Add model configuration, evidence context and strict response validation
`agent2.ts`	Hospital/resource eligibility logic	Keep operational filtering outside vector search
`agent3.ts`	Routing and time-to-care ranking	Consume confirmed requirements and route data, not RAG opinions about hospital rank
`db/store.ts`	Runtime state primarily held in JavaScript Maps	PostgreSQL schema existence does not mean business writes are durable
`db/index.ts` and migrations	PostgreSQL pool, connectivity check and SQL schema	Establish real repositories and transactions before claiming durable coordination
Current map files	`LiveMap.tsx`, `MapPanel.tsx`, `google-routes.ts` and routing contracts are present	Assess existing work; do not blindly recreate the earlier map plan
`Simulation3DView.tsx`	React Three Fiber / Drei / Three.js simulation	Keep optional, lazy-loaded and visibly simulated

2.1 Specific gaps the implementation must address

1. No retrieval, embedding store or approved-document ingestion path was found in the inspected agent flow.
2. Current Agent 1 generation parses JSON and checks that selected fields are arrays; it does not establish citation validity or constrain every returned tag against an approved capability registry.
3. Current fixed confidence values such as 0.95 and 1.0 are not calibrated probabilities. Do not present them as clinical certainty or evidence quality.

4. A Promise timeout can stop waiting without cancelling the external request. Add provider-supported cancellation or otherwise account for late responses and costs.
5. The interpreter comment describes input sanitization, but comments do not prove isolation against prompt injection. Implement and test actual boundaries.
6. Rules in the demonstration code contain medical assumptions. Preserve compatibility for demo testing while having an authorized domain reviewer decide the approved rule pack; do not automatically promote current rules into a clinical knowledge source.
7. The separate coordinator briefing call can generate advice outside Agent 1. Keep it to summarizing validated operational fields, or subject additional clinical claims to the same evidence controls.
8. Database health checks and migrations do not prove transactional business persistence. Inventory concurrency and recovery require explicit implementation and tests.

3. Recommended technology stack and why it fits

"Better" below means a better fit for this repository and initial scale, not universally superior technology.

Technology	How GoldenHour uses it	Why keep or choose it	Tradeoff
React + TypeScript	Forms, maps, evidence review, role dashboards	Existing skills and components; typed UI state	Types do not validate external runtime data
Vite + Tailwind	Development/build pipeline and shared design tokens	Fits the current application; fast UI iteration	Public frontend environment values are not secrets
Fastify + Zod	Authenticated API endpoints and DTO validation	Fits current server; clear request/output boundaries	Authorization and semantic checks still need explicit code
PostgreSQL	Cases, inventory, reservations, knowledge metadata and audits	Transactions and constraints fit operational consistency	Requires actual repository migration from the in-memory store
pgvector extension	Embeddings stored with chunk metadata	One database to operate initially; SQL filtering and relational joins	Search tuning and index memory become important at scale
PostgreSQL full-text search	Exact terms and lexical relevance	Complements semantic retrieval without another search service	Language configuration and acronym handling need evaluation
Google GenAI SDK	Embedding calls and grounded JSON generation	Already installed; avoids unnecessary provider plumbing	Model availability, privacy, cost and rate limits must be confirmed
Private object storage	Original approved files and extraction artifacts	Separates large files from query tables; supports versioned provenance	Needs access control, retention and backup configuration

Technology	How GoldenHour uses it	Why keep or choose it	Tradeoff
PostgreSQL-backed worker	Durable ingestion jobs, retries and leases	Avoids adding Redis solely for the initial document pipeline	A high-volume workload may justify a dedicated queue later
Socket.IO	Case, inventory and position updates	Existing transport and frontend subscriptions	Rooms, reconnects and role changes need authorization tests
Google Maps + Routes	Basemap, route geometry and travel estimates	Aligns with current map integration direction	Billing, restrictions and attribution; no automatic GPS feed
Vitest + Testing Library	Unit, API and UI regression tests	Already available in the monorepo	Add real database integration checks; mocks alone are insufficient
Docker + Nginx	Deployment packaging and reverse proxy	Existing deployment artifacts	Build/runtime configuration and migrations must be verified

pgvector supports exact search and approximate indexes such as HNSW; filtering can affect approximate-search recall. Start with exact search on a small corpus and benchmark before adding an approximate index. These capabilities are described in the official pgvector documentation.

PostgreSQL full-text search provides token processing, matching and ranking. Use it for terms such as an exact capability name while vector retrieval handles paraphrases. See PostgreSQL full-text search.

3.1 Alternatives and the decision rule

- **Qdrant or another dedicated vector service:** Consider if independent scaling, specialized filtered search or existing team infrastructure justifies an additional service. It does not remove PostgreSQL's role in reservations. Do not add it merely to use the label “vector database”.
- **Managed PostgreSQL with pgvector:** A good deployment option if the chosen host supports the extension, required version, region, backups and connection limits. The host is a user-dependent decision.
- **Managed file-search/RAG product:** May reduce ingestion work, but verify control over version publication, deletion, access filters and citation fidelity before selecting it.
- **Fine-tuning:** Useful for some behavior/style problems, but not the first solution for changing reference documents or traceable citations. RAG retrieves knowledge at request time; it does not train new model weights.
- **LangChain or a similar orchestration framework:** Optional. Start with typed services because the proposed sequence is bounded and already fits Fastify. Add a framework only if it measurably simplifies tested workflow requirements.
- **A separate Python AI service:** Not initially required. Keep runtime orchestration in TypeScript; use an isolated extraction/OCR worker only where supported file formats demand it.

3.2 Embedding and model selection

Use separate configurable generation and embedding model IDs. A reasonable text-only baseline to evaluate is `gemini-embedding-001`, with a supported reduced dimension such as 768; verify account availability, requested output dimension and quality before fixing the schema. Gemini documents embedding models, task types and output dimensions in its embedding guide.

Record provider, model ID, dimension, task configuration, normalization policy and embedding version together. The query and document embeddings must use compatible settings and the same embedding space. Do not mix models just because vectors have the same length.

Choose the generation model using an account-level smoke test and the evaluation dataset. The model string currently in `gemini.ts` is evidence of configuration in source, not proof of availability in the user's account. Do not hardcode a new model based only on this plan.

4. Target architecture and responsibilities

4.1 Knowledge publication path

```
Authorized uploader
  -> private original file + source metadata
  -> queued extraction job
  -> heading-aware chunks + page/section locators
  -> embedding generation + lexical index
  -> automated checks + reviewer preview
  -> approved corpus release
  -> retrieval-visible documents
```

Uploading a file must not immediately make it authoritative. Publication requires review and an atomic release switch after all required chunks and embeddings are ready.

4.2 Case assessment path

```
Paramedic observations
  -> authentication + case access + validated observation snapshot
  -> minimized retrieval query
  -> authorized current corpus + hybrid retrieval
  -> selected evidence context
  -> Agent 1 structured generation
  -> schema, tag, citation and evidence checks
  -> proposal with evidence / explicit abstention
  -> clinician review and confirmation
  -> Agent 2 live eligibility
  -> Agent 3 time-to-care ranking + map route
  -> existing dispatch / acceptance / reservation lifecycle
```

4.3 Agent boundaries

Component	Allowed responsibility	Must not decide
Agent 1 RAG	Propose allowed requirement tags and evidence-linked reasons; flag missing information	Actual bed availability, destination acceptance, autonomous treatment or dispatch
Clinician	Confirm/edit requirements and review evidence	Treat a similarity score as clinical certainty
Agent 2	Check current structured capabilities, staff and resource eligibility	Infer capacity from a protocol PDF
Agent 3	Rank eligible destinations by defined time-to-care logic	Override clinician requirements using generated text
Reservation service	Atomically allocate/release resources with version checks	Delegate writes or transaction decisions to an LLM
Map adapter	Display authorized positions and provider routes	Prove a hospital can receive a patient

5. Required user inputs and API questionnaire

Fill this section before the dependent implementation stage. Defaults permit scaffolding and tests; they do not substitute for required source approval, credentials or a deployment decision. **Record secret locations or secret-manager references, not actual keys/passwords, in this document.**

5.1 Product, scope and ownership

ID	Question to answer	Suggested starting answer	Needed by
P01	Is this a hackathon demo, internal pilot or intended clinical deployment?	Synthetic demo	Stage 1
P02	Which country/region, hospital network and protocol jurisdiction apply?	Explicitly named demo jurisdiction	Stage 1
P03	Who owns product decisions and who approves knowledge content?	Named product owner and domain reviewer	Stage 1
P04	Which scenarios and patient populations are in scope?	Existing scenarios, reviewed individually	Stage 1
P05	Which languages must retrieval and UI support?	English first	Stage 1
P06	Is Agent 1 limited to resource requirements, or are other outputs requested?	Requirements and cited reasons only	Stage 1
P07	Is manual clinician entry allowed when RAG abstains?	Yes, with provenance and confirmation	Stage 1
P08	Is tracking single-unit or fleet-wide?	Existing single-unit demo first	Stage 3
P09	What latency and operating-cost targets are acceptable?	Evaluate proposed targets in Stage 4	Stage 1
P10	Who approves release and maintains the system afterward?	Named engineering and knowledge owners	Stage 4

5.2 Documents, data and approval

ID	Question to answer	Suggested starting answer	Needed by
D01	Which exact documents may be indexed, and where are they supplied?	Small explicitly approved set	Stage 1
D02	Do you have permission to process/store/display each document and excerpts?	Owner records permission	Stage 1
D03	What file formats, page counts and total volume are expected?	Text PDF/Markdown first; record actual totals	Stage 1
D04	Are any PDFs scanned or table-heavy, requiring OCR/manual extraction review?	Inspect before choosing parser	Stage 2
D05	Who approves effective dates, superseded versions and source conflicts?	Named domain reviewer	Stage 1
D06	Are sources global, regional, organization-specific or hospital-private?	Explicit scope on every source	Stage 1
D07	What is the approved capability tag registry and synonym mapping?	Review existing enums/tags	Stage 1
D08	Are all patient inputs synthetic? If not, what data may leave the environment?	Synthetic only for initial delivery	Stage 1
D09	What retention/deletion rules apply to originals, chunks, embeddings and traces?	Owner-defined durations; no guessed policy	Stage 1
D10	Who will label expected evidence and proposals for evaluation?	Domain reviewer plus engineering owner	Stage 2
D11	May uploaded documents contain external links, attachments or executable content?	No automatic following/execution	Stage 2
D12	How often must knowledge be reviewed and updated?	Per-source review date plus manual revocation	Stage 2

5.3 APIs, credentials and infrastructure

ID	Required input	What to supply without exposing secrets	Needed by
A01	Generation provider/account	Provider, project, region, enabled model IDs, secret reference	Stage 1
A02	Embedding provider/account	Model ID, supported dimensions/task types, quota, secret reference	Stage 1
A03	Provider processing/retention constraints	Approved account/service settings and allowed data categories	Before external data calls
A04	PostgreSQL host	Host/service, version, extension support, TLS, secret reference	Stage 1
A05	Object storage	Bucket/container, region, private access method, retention/versioning	Stage 2
A06	OCR provider, if needed	Approved service/local tool, languages, limits, secret reference	Stage 2 if scanned files
A07	Deployment target	Local Docker, VM or managed hosting; environments and domains	Stage 1
A08	Worker execution	Persistent process/job platform, concurrency and restart behavior	Stage 2
A09	Monitoring destination	Logs/metrics service, redaction rules and alert owner	Stage 4
A10	Backup and recovery	Backup frequency, recovery-point/time targets, restore owner	Stage 4
A11	Maps browser credential	Restricted key reference, allowed domains and map ID	Map verification
A12	Routes server credential	Restricted backend key reference and billing/quota setup	Map verification
A13	GPS source	Browser/device feed, assigned unit and permission flow	Live tracking
A14	Monthly budget and quotas	Separate caps/alerts for generation, embeddings, storage and maps	Before paid calls
A15	Authentication and tenants	Existing demo roles versus real identity provider and tenant model	Stage 1

No separate vector-service API key is required for pgvector: it is accessed through PostgreSQL credentials. If a dedicated vector provider is chosen instead, supply its endpoint, region, credential reference, collection configuration and deletion/backup requirements.

5.4 Fill-in decision record

```
Project mode:
Deployment region / jurisdiction:
Product owner / knowledge reviewer:
Approved scenarios and languages:
Approved document list and permissions location:
Tenant / document-access policy:
Patient data permitted for external processing:
Generation provider / model / secret reference:
Embedding provider / model / dimension / secret reference:
PostgreSQL provider / version / pgvector version:
Object storage location / private access:
OCR needed? Tool/provider:
Expected documents / chunks / assessments per day:
Monthly budget / provider quotas:
Retention / deletion / backup requirements:
Map credentials references / map ID / domains:
GPS source and unit assignment:
Release approver / ongoing maintenance owner:
Unresolved decisions and their owners:
```

6. Stage 1 - Confirm scope, persistence and shared contracts

Goal: Establish a trustworthy data boundary before adding model behavior. **Inputs:** P01-P07, P09, D01-D09, A01-A04, A07 and A15. **Deliverable:** Approved architecture decisions, database plan, contracts and executable provider smoke-test specification.

6.1 Inventory and decision work

1. Capture current routes, agent signatures, local map work and test expectations. Do not overwrite in-progress changes.
2. Inventory approved documents and capability tags. Assign owners, jurisdiction, language and effective dates.
3. Decide demo-only versus durable operational scope. For a full persistent implementation, migrate business state from JavaScript Maps into repositories backed by PostgreSQL.
4. Define `CaseRepository`, `HospitalRepository`, `AllocationRepository`, `KnowledgeRepository` and `AuditRepository` interfaces. Keep an explicit in-memory adapter for isolated tests/demo use.
5. Prohibit automatic fallback to in-memory writes when the production database fails; return an observable service error instead.
6. Configure generation and embedding interfaces independently, then validate actual model access with synthetic input.

6.2 Operational persistence migration

- Trace every read/write through `store.ts` and the case, hospital, dispatch and admin routes.
- Migrate one coherent transaction boundary at a time rather than dual-writing independent in-memory and SQL states.
- Persist resource checks and reservation writes in one transaction. Lock affected rows in a stable order, recheck versions/capacity, and roll back all affected allocations on failure.

- Store domain changes and outbox events in the same transaction. Publish after commit and make consumers idempotent.
- Preserve existing allocation semantics: standby B/C do not reserve; arrival and release remain distinct.
- Verify restart recovery and two-process contention before claiming persistence. A single-process unit test is not sufficient.
- Keep RAG/embedding network calls outside reservation transactions and row locks.

If only the RAG demo is implemented first, knowledge may be durable while case state remains explicitly labelled volatile. The presentation must state this limitation; it must not describe full production persistence.

6.3 Proposed knowledge schema

Table	Essential fields	Purpose
`knowledge_documents`	ID, tenant/scope, title, owner, authority, source URI, classification	Stable logical document identity
`knowledge_document_versions`	ID, document ID, version, file hash, storage key, language, jurisdiction, effective/expiry/review dates, approval state	Immutable source version and lifecycle
`knowledge_chunks`	ID, version ID, ordinal, text, heading path, page range, source offsets, token count, content hash, lexical vector	Traceable retrieval units
`embedding_sets`	ID, provider, model, dimension, normalization, parser/chunker versions, status	Compatible embedding configuration
`knowledge_embeddings`	Chunk ID, embedding-set ID, vector, created time	Searchable representations
`corpus_releases`	ID, scope, embedding-set ID, published time, publisher, manifest hash	Reproducible approved snapshot
`corpus_release_versions`	Release ID, document-version ID	Explicit release membership
`ingestion_jobs`	ID, source version, state, attempts, lease expiry, progress, error code	Durable asynchronous ingestion
`rag_runs`	ID, case/assessment version, release ID, query hash, model/prompt versions, status, durations, token usage	Reproducible run metadata
`rag_evidence`	Run ID, claim ID, chunk ID, source locator, retrieval ranks, validation status	Links output claims to retrieved evidence

Keep patient observations in access-controlled case/assessment records, not in the reusable knowledge index. Store only the minimum query/trace content needed under the chosen retention policy.

For a 768-dimensional baseline, use a matching `vector(768)` column and check generated vector length. If another dimension is chosen, use a separately versioned table/index or controlled migration. Never silently truncate or compare incompatible embeddings. With pgvector, HNSW indexing supports `vector` up to 2,000 dimensions and `halfvec` up to 4,000; verify the deployed extension and chosen operator class before implementation. See pgvector index documentation.

6.4 Shared DTOs and service boundaries

- Add proposed `packages/contracts/src/knowledge.ts` and `rag.ts`, exported from `index.ts`.
- Extend the proposal contract with `proposalId`, `assessmentVersion`, `mode`, `status`, `claims`, `citations`, `missingInputs`, `corpusReleaseId` and provenance versions.
- Use `mode` values such as `RAG`, `RULES_FALLBACK` and `MANUAL`; use explicit status values such as `SUPPORTED`, `PARTIALLY_SUPPORTED`, `ABSTAINED` and `FAILED`.
- Keep existing requirement arrays for compatibility while linking each proposed tag/reason to a stable claim ID.
- Define evidence coverage separately from confidence. Retrieval similarity is not probability of correctness.
- Add maximum lengths and array sizes. Unknown tags and unsupported fields must fail validation or be explicitly surfaced for manual review, never silently admitted.

6.5 Acceptance gate

- Architecture decisions and input owners are recorded; unresolved required inputs are visible.
- SQL migration and rollback are tested on a disposable database with the selected extension.
- Durable mode passes restart and concurrent reservation tests, or the release is explicitly scoped as a volatile demo.
- DTO validation rejects invalid modes, unsupported tags and malformed evidence references.
- No credentials are committed, returned to the browser or written into source metadata.

7. Stage 2 - Build ingestion, publication and retrieval

Goal: Make approved knowledge searchable and traceable before connecting generation. **Inputs:** Approved source files, access policy, object storage, embedding access, OCR decision and reviewer. **Deliverable:** A published test corpus and a retrieval evaluator with no patient-facing generation yet.

7.1 Ingestion pipeline

1. Accept uploads only from an authorized knowledge role. Enforce file size/type/page limits and inspect actual file signatures.
2. Store the original privately with a hash and immutable version ID. Avoid public object URLs.
3. Create a durable job. A worker claims it with a lease, reports progress and retries only transient failures with bounded backoff.
4. Parse text PDFs/Markdown first. Add DOCX or OCR only when the input inventory requires them. Do not assume PDF reading order is correct.
5. Preserve headings, paragraph boundaries, list structure, units, negation and table associations. Flag extraction failures, missing pages and low-quality OCR for review.
6. Start with heading-aware chunks around 400-700 tokens and limited overlap around 50-100 tokens. These are tuning candidates, not universal values. Keep a rule and its exception together; treat tables as structured units.
7. Attach document/version/page/section locators to every chunk. Do not invent page numbers for formats without pages; use heading/paragraph references.

8. Deduplicate repeated headers and identical chunks without losing all source associations. Use version/content hashes for idempotent jobs.
9. Generate document embeddings with the chosen model and appropriate document task configuration, validate finite values/dimension, then create lexical indexes.
10. Show a reviewer the original, extracted text, chunks and sample search results. Publish only after approval and completeness checks.

7.2 Publication, updates and deletion

- Keep draft, processing, review, published, superseded and revoked states explicit.
- Publish a release manifest atomically. An assessment uses one release throughout a run.
- Re-index into a new embedding set when model, dimension or chunking changes. Evaluate it before switching the active release; retain a compatible rollback target.
- Treat urgent revocation differently from normal supersession: deny revoked sources immediately, including in-flight runs before serving results.
- Keep old evidence references for permitted audits, but do not allow retired documents to influence new assessments.
- Deletion must cover originals, extraction artifacts, chunks, embeddings, authorized caches and traces according to policy. Define backup expiry separately; do not promise immediate deletion from immutable backups.

7.3 Hybrid retrieval algorithm

1. Build a concise query from permitted observations and known/missing fields. Remove direct identifiers not required for retrieval.
2. Determine tenant, region, language, population and active approved release from authenticated context. The model must not supply access-control filters.
3. Apply access, publication and applicability filters to both lexical and semantic candidate sets before either contributes evidence to the prompt.
4. Retrieve an initial candidate pool, for example 20 semantic and 20 lexical results. Merge by chunk identity and fuse ranked lists with a documented method such as reciprocal rank fusion.
5. Deduplicate near-identical passages, keep necessary adjacent context, and select approximately 4-8 evidence chunks within a bounded context budget.
6. Start without a paid reranker. Add one only if held-out retrieval evaluation shows a meaningful improvement and its processing policy is approved.
7. Use a quality gate calibrated on labeled examples. Low similarity, no current evidence, source conflict or wrong-population evidence must lead to abstention/review, not forced completion.
8. Return chunk IDs, source versions, locators and retrieval metadata. Do not let the generation model invent citation URLs.

With a small corpus, exact vector search provides a useful relevance baseline. At larger scale, evaluate HNSW against that baseline under real tenant/date filters. Approximate search followed by filtering may return too few candidates; tune or use exact filtered search rather than removing access filters. Never compare arbitrary scores from different retrieval methods as though they shared one scale.

7.4 Proposed endpoints

Endpoint	Authorized role	Behavior
`POST /api/v1/knowledge/documents`	Knowledge administrator	Validate metadata/upload intent and create draft version
`POST /api/v1/knowledge/versions/:id/ingest`	Knowledge administrator	Queue idempotent ingestion
`GET /api/v1/knowledge/jobs/:id`	Authorized operator	Read progress/error state
`POST /api/v1/knowledge/releases`	Knowledge reviewer	Publish approved complete manifest
`POST /api/v1/knowledge/versions/:id/revoke`	Knowledge reviewer	Revoke with reason and audit record
`POST /api/v1/knowledge/search-preview`	Knowledge reviewer	Evaluate retrieval without clinical generation
`GET /api/v1/knowledge/chunks/:id/source`	Authorized source reader	Resolve approved excerpt/original access

Introduce knowledge permissions explicitly; do not assume every current admin user should automatically publish reference material. Use request limits, idempotency keys and expected-version checks for mutation endpoints.

7.5 Acceptance gate

- Re-ingesting the same source/version does not duplicate searchable chunks.
- Every result has a valid original-source locator.
- Draft, expired, revoked and unauthorized content cannot enter a generation context.
- Worker interruption/restart resumes safely without publishing a partial corpus.
- Retrieval is evaluated independently on paraphrases, exact tags, missing evidence, conflicting sources and access-boundary cases.

8. Stage 3 - Integrate Agent 1 RAG and evidence review UI

Goal: Replace unsupported free-form interpretation with a bounded evidence-based proposal flow. **Inputs:** Published corpus, retrieval evaluation, approved model access and fallback policy. **Deliverable:** End-to-end assessment proposals with validated citations and clinician review.

8.1 Interpreter execution

1. Keep `runAgent1Async` as the entry point; select the interpreter through a feature flag or configuration.
2. Create an immutable observation/assessment version and run ID before retrieval.
3. Retrieve from the selected corpus release using the authenticated applicability scope.
4. If evidence is insufficient, return an explicit abstention or separately labelled approved rules fallback. Do not call an unconstrained LLM and present it as RAG.
5. Construct a prompt with separately delimited instructions, observations and untrusted evidence. Retrieved documents and patient notes cannot modify system instructions or request tools.
6. Include only allowed tag definitions and bounded evidence excerpts. Never include secrets, unrestricted database access, or executable tools in Agent 1.
7. Request structured output matching the shared response schema. Gemini supports schema-constrained output, but valid JSON does not prove medical or semantic correctness. See Gemini structured outputs.
8. Parse with Zod, validate allowed tags, check claim-to-evidence references, and ensure each citation belongs to this run's authorized retrieved set.
9. Verify that cited excerpts support the actual claim, not merely that the ID exists. Use conservative checks and reviewer evaluation; an optional secondary model check is evidence of a check, not a guarantee.
10. Surface unsupported or contradictory claims explicitly. Do not silently drop required fields and return an apparently complete successful proposal.
11. Persist the run/proposal and provenance. Apply it only if the observation version is still current; discard late results from superseded assessments.
12. Let the clinician confirm/edit requirements. Record the proposal version, final requirements, clinician identity and override rationale where required. Confirmation remains a separate action.

8.2 Output contract example

```
{
  "proposalId": "proposal-demo-001",
  "assessmentVersion": 3,
  "mode": "RAG",
  "status": "PARTIALLY_SUPPORTED",
  "proposedFullCareNeeds": ["EMERGENCY_DEPT"],
  "proposedStabilizationNeeds": [],
  "claims": [
    {
      "id": "claim-1",
      "capabilityTag": "EMERGENCY_DEPT",
      "reason": "Illustrative claim; replace with reviewed evidence.",
      "evidenceIds": ["chunk-example-12"]
    }
  ],
  "missingInputs": ["Required observation is not recorded"],
  "citations": [
    {
      "chunkId": "chunk-example-12",
      "documentVersionId": "document-example-v2",
      "section": "Illustrative section",
      "page": 4
    }
  ],
  "corpusReleaseId": "release-example-2"
}
```

This is a contract illustration, not a clinical recommendation or a real source citation. Server-resolved metadata must supply the actual title, version and locator. Existing confirmation rules may require nonempty requirement lists; partial output must not automatically pass them.

8.3 Failure behavior

Failure	Required behavior
Embedding provider unavailable	Optional explicitly labelled lexical-only retrieval if evaluated; otherwise abstain
Vector/knowledge database unavailable	No fabricated citations; approved rules fallback or manual workflow
No applicable published evidence	“No approved supporting source found”; request review
Conflicting sources	Show conflict and source versions; apply only a pre-approved precedence rule
Model timeout or invalid JSON	Bounded retry only if budget allows; then labelled fallback/abstention
Unsupported tag or invented citation	Reject affected proposal; never silently promote it
Assessment changed during generation	Discard stale result and offer/restart current assessment run
Source revoked during generation	Revalidate release eligibility and reject affected evidence
Prompt-injection text in source	Treat as data; no tool execution or access-policy changes

The existing demonstration rules are not automatically a trusted fallback. Approve and version the rule pack first. If no approved fallback exists, retain manual clinician entry and mark automation unavailable.

8.4 UI changes

- Extend the Ambulance assessment panel with source-linked reasons and a clear RAG/rules/manual mode label.
- Show a concise evidence summary, missing inputs and unresolved conflicts before the confirmation action.
- Open sources in an accessible drawer with title, version, page/section and relevant excerpt; allow authorized access to the original.
- Keep source freshness separate from GPS freshness and hospital inventory freshness.
- Replace fixed “95% confidence” presentation with factual status such as “3 claims have linked evidence; 1 needs review”. Evidence coverage is not certainty.
- Add a Knowledge area for authorized users: source list, processing jobs, extraction preview, approval, publication and revocation history.
- Reuse the light teal/white design and shared components in the existing map/UI work. Avoid adding a second visual system.
- Keep source inspection and query review separate from dispatch actions; clicking a citation cannot change requirements or reserve resources.

8.5 API integration and audit

Preserve the existing assessment submission route where possible. Add a read endpoint such as `GET /api/v1/cases/:caseId/assessments/:version/evidence`, authorized against the case and document scope. For asynchronous execution, return a run ID and expose `GET /api/v1/rag/runs/:id` with explicit pending/completed/failed states.

Record release ID, document versions, retrieved chunk IDs/ranks, model and prompt versions, validation result, duration, token usage and clinician edits. Do not store hidden chain-of-thought or claim that a generated explanation is the model's internal reasoning. Store the evidence and explicit decision rationale needed for review.

8.6 Acceptance gate

- Every displayed source link resolves to evidence retrieved for the same authorized run.
- Unsupported claims, invalid tags and no-evidence cases cannot silently appear as supported output.
- A changed assessment never receives an older run's proposal.
- Agent 1 cannot dispatch, reserve, change stock or override user permissions.
- Clinician confirmation and the existing Agent 2/3 workflow still pass regression tests.

9. Stage 4 - Evaluate, harden, deploy and prepare the demonstration

Goal: Establish measurable quality, predictable failure behavior and a credible project explanation. **Inputs:** Reviewed evaluation set, budgets, deployment configuration and release owner. **Deliverable:** Evaluation report, tested deployment, rollback procedure and presentation walkthrough.

9.1 Evaluation dataset and metrics

Start with approximately 60-100 synthetic cases authored/reviewed for the scoped scenarios. Reserve a held-out subset before tuning. Include paraphrases, exact terms, missing observations, contradictory notes, unrelated documents, wrong jurisdiction, outdated sources, OCR corruption and attempted prompt injection. Size and thresholds are proposed engineering targets, not evidence of clinical validation.

Metric	What it measures	Proposed gate
Retrieval recall@k	Whether required labeled evidence appears in the retrieved set	Target at least 90% on held-out scoped examples; report by scenario
Citation resolution	Whether IDs/versions/locators resolve correctly	100% in acceptance tests
Claim support	Whether a reviewer agrees the cited text supports each claim	No known unsupported high-impact claims in release set; report numerator/denominator
Requirement precision/recall	Match against reviewer-approved capability sets	Reviewer-defined gate by scenario, including critical omissions
Abstention quality	Appropriate refusal when evidence is absent/inapplicable	All mandatory no-evidence fixtures abstain or clearly mark fallback
Authorization	Whether unauthorized evidence can be retrieved or viewed	Zero leakage in access-boundary tests
Lifecycle consistency	Existing reservation/diversion behavior	All relevant regression tests pass
Latency	Retrieval and full proposal time under realistic load	Initial targets: retrieval p95 <= 1 second; full proposal p95 <= 8 seconds

Define a bounded total deadline, initially around 10 seconds subject to measurement, and show a pending state. The current 2.5-second model-only timeout is not automatically a suitable end-to-end RAG budget. Fail transparently when the budget expires.

9.2 Tests and checks

- Unit tests: chunking, locator preservation, tag allowlist, vector dimensions, query minimization and citation validation.
- Real database integration: extension migration, filtered retrieval, publication transactions, revoked-source exclusion, two-process allocation contention and restart recovery.
- Provider contract tests: mock successful/malformed embedding responses, quota limits, timeout and structured-output failures.
- UI tests: source drawer, unsupported state, partial proposal, manual review, keyboard interaction and role restrictions.

- Failure drills: worker crash mid-ingestion, embedding service outage, database outage, stale assessment, duplicate job, source revocation and failed rollback.
- Existing checks: `pnpm test`, `pnpm typecheck`, `pnpm build`. Current placeholder lint scripts are not a substantive quality check.
- Manual smoke tests: a few synthetic generation/embedding requests against restricted real credentials, separate from routine CI.
- Visual review: ambulance, hospital, dispatch and knowledge screens at desktop/mobile widths, including empty/loading/error states.

9.3 Cost, scale and observability

Use the selected provider's current rates during implementation; do not promise a fixed monthly price from this document.

```
Initial embedding usage = total indexed source tokens
Ongoing embedding usage = changed-source tokens + query tokens
Generation usage = assessments x average input/output tokens
Infrastructure = database + private files + worker + backups + logs
Map usage = map loads + route requests, tracked independently
```

A 768-dimensional float32 vector uses roughly 3 KB for raw values before row/index overhead. At 100,000 chunks that is roughly 300 MB of raw vectors, with additional storage for text, metadata and indexes. Measure the actual database size rather than treating raw-vector arithmetic as a capacity guarantee.

Track ingestion lag, job failures, retrieval latency, source age, no-evidence rate, validation failures, fallback rate, model tokens, and per-assessment cost. Log identifiers and aggregate timings by default; raw notes and excerpts need explicit retention/access rules. Add provider concurrency caps and per-user/case rate limits.

9.4 Deployment and rollback

1. Pin a PostgreSQL/pgvector combination verified in staging. The current plain `postgres:16-alpine` image does not itself establish pgvector availability.
2. Apply additive migrations and verify rollback/restore on a disposable environment first.
3. Deploy the ingestion worker separately from request handling; make job leases/retries safe across restarts.
4. Keep the knowledge library private and credentials in environment-specific secret configuration.
5. Enable RAG in shadow mode on synthetic cases: record results without changing the visible workflow.
6. Compare with labeled cases and the current baseline, then enable evidence review for a controlled demo.
7. Record the corpus release, embedding set, prompt version, model ID and application build together.
8. Roll back through a feature flag and a compatible prior corpus release. Never roll back a revoked source into use.
9. Test backup restoration including source objects, metadata and embeddings. If source text survives, embeddings are rebuildable; restoration time still matters.
10. Document outstanding clinical, privacy and operational review separately before any real-world use. This implementation plan does not claim those reviews have occurred.

9.5 Final definition of done

- Approved sources have owners, locators, versions, effective dates and controlled publication.
- Agent 1 retrieves before generating and links claims to validated evidence.
- Empty/conflicting/unauthorized evidence produces explicit review or abstention behavior.

- Live hospital status and reservations remain authoritative relational operations.
- The frontend displays citations, missing inputs, source mode and clinician confirmation clearly.
- Evaluations and failure tests are recorded with actual results, not just target numbers.
- The deployment and presentation accurately distinguish existing, newly implemented, simulated and unverified behavior.

10. File-level implementation map

All entries are proposed future work. This documentation delivery does not create these modules or modify existing ones.

Area	Existing files to review/extend	Proposed additions
Shared contracts	`packages/contracts/src/case.ts`, `index.ts`	`knowledge.ts`, `rag.ts`
Agent integration	`agent1.ts`, `interpreter.ts`, `gemini.ts`	`rag-interpreter.ts`, typed generation adapter
Retrieval	API module registration	`modules/knowledge/retriever.ts`, `query-builder.ts`, `evidence-validator.ts`
Ingestion	API config and package scripts	`modules/knowledge/ingestion.ts`, `chunker.ts`, `extractors/`, `worker.ts`
Provider adapters	Existing Google client/config	`modules/knowledge/embedding-provider.ts`, private storage adapter
Persistence	`db/index.ts`, `db/store.ts`, `db/migrate.ts`	Knowledge and operational repositories; next-numbered SQL migration
API routes	`modules/cases/routes.ts`, auth/registration	`modules/knowledge/routes.ts`, run/evidence endpoints
Web	`AmbulancePage.tsx`, `AdminPage.tsx`, `api-client.ts`	`EvidenceDrawer.tsx`, `KnowledgePage.tsx`, ingestion status views
Tests	Existing Agent 1, workflow and UI suites	Retrieval, corpus lifecycle, auth, citation and worker integration suites
Deployment	`.env.example`, Compose/Docker files, README	Documented worker launch and extension setup

10.1 Proposed configuration inventory

```
AGENT1_MODE=rag
RAG_ENABLED=false
DATABASE_URL=<secret reference resolved by deployment>
GENERATION_PROVIDER=google
GENERATION_MODEL=<verified account-supported model>
EMBEDDING_PROVIDER=google
EMBEDDING_MODEL=<verified text embedding model>
EMBEDDING_DIMENSIONS=768
GEMINI_API_KEY=<server secret>
KNOWLEDGE_STORAGE_PROVIDER=<local-private-or-approved-object-store>
KNOWLEDGE_BUCKET=<private location if applicable>
KNOWLEDGE_ACTIVE_RELEASE=<published release identifier>
INGESTION_CONCURRENCY=2
RAG_CONTEXT_TOKEN_LIMIT=5000
RAG_TIMEOUT_MS=10000
RAG_ALLOW_APPROVED_RULES_FALLBACK=false
```

Values are proposed starting settings. Confirm them with provider limits and evaluation. Validate required production configuration at startup rather than silently substituting development defaults. Never expose generation, embedding, database or storage secrets through `VITE_` variables.

10.2 Dependency sequence and planning estimate

Stage	Depends on	Rough engineering effort after inputs are ready
1: scope, persistence, contracts	Required decisions and DB/provider access	4-7 working days; persistence migration may extend this
2: ingestion and retrieval	Approved sources and Stage 1 schema	4-7 working days
3: Agent 1 and UI	Published corpus and retrieval baseline	4-6 working days
4: evaluation and rollout	End-to-end implementation and reviewers	4-7 working days

These are planning ranges for one engineer familiar with the repository, not commitments. OCR complexity, a large persistence refactor, new authentication, missing documents or review delays can materially increase effort. A small synthetic proof of concept can omit production rollout, but must state the omitted scope.

11. How to explain the project to anyone

11.1 Simple analogies

- PostgreSQL is the live ledger: who has which resource, what is reserved and what changed.
- The document library is the approved reference shelf.
- An embedding is a numeric search representation of a passage, not a truth score or encrypted copy.
- Vector retrieval finds passages with related meaning; keyword search finds precise terminology.
- RAG opens the relevant reference pages before drafting an explanation.
- The clinician is the confirmation authority; the agent supplies a proposal and evidence.
- The map estimates the journey; hospital data determines whether the destination can support the confirmed needs.

11.2 Two-minute technical explanation

"The frontend is React and TypeScript, with shared Zod contracts so the server and interface agree on data shapes. Fastify exposes authenticated APIs. Operational records belong in PostgreSQL because resource reservations require transactional consistency.

For Agent 1, we add a controlled knowledge pipeline. Approved documents are extracted into traceable chunks. An embedding model converts those chunks into vectors stored with pgvector, while PostgreSQL full-text search handles exact terms. At assessment time, the service retrieves applicable evidence, sends only selected passages and observations to the generation model, and validates structured output and citations.

The clinician reviews the proposal. Agent 2 checks current hospital resources; Agent 3 ranks eligible destinations by time-to-care. Socket.IO distributes authorized updates. The map shows real geography and route estimates, while simulated movement remains explicitly labelled. The value is coordination with traceability, not autonomous clinical decision-making."

11.3 A demonstration walkthrough

1. Open Knowledge and show a clearly marked synthetic/approved demonstration source, its version and publication status.
2. Show a prepared synthetic assessment with one missing observation. Explain that missing does not mean normal.
3. Run Agent 1. Open a reason and its exact document section; distinguish source retrieval from model generation.
4. Change or remove supporting evidence in a separate test corpus and show abstention, rather than claiming the system always produces an answer.
5. Confirm requirements as the clinician. Show Agent 2's current capability/resource checks and Agent 3's time-to-care components.
6. Select a destination and demonstrate the existing acceptance/reservation flow. The map selection itself performs no reservation.
7. Show an audit record with assessment version, source release, model version and clinician action.
8. Demonstrate a provider outage or stale GPS state and explain the visible fallback mode.

Do not call the walkthrough a clinical validation study. Use actual measured evaluation results when discussing quality.

11.4 Questions judges or teammates may ask

Why not simply ask an LLM? A direct prompt does not guarantee current approved evidence. Retrieval makes sources available at request time, and citation validation makes the proposed basis inspectable. Errors remain possible and are evaluated explicitly.

Why pgvector rather than putting everything in a vector database? This project already has relational entities and transactions. pgvector adds semantic retrieval while retaining SQL metadata controls. Live inventory and reservations still require exact relational operations.

Does RAG train the model on our files? No. This design retrieves passages and includes them in the request context. It does not fine-tune the model. Provider processing/retention settings are a separate account decision.

Does a citation prove the answer is correct? No. It proves a source can be inspected only after the link is validated. The claim still needs to be supported by that source and appropriate to the case.

Are all three agents LLMs? No. "Agent" names describe responsibilities. Agent 1 uses retrieval and generation; eligibility, ranking and resource allocation are better kept in explicit, testable logic.

Why not choose the nearest hospital? The defined objective includes eligibility and time-to-care. A short drive may not be useful if the required resources are unavailable or receiving delay is long.

Does Google Maps provide the ambulance's location? No. The application needs an assigned GPS/device feed. The basemap, route estimates and vehicle position are separate inputs.

What happens when the reference library is outdated? Version dates, review ownership, supersession and revocation control which sources can be used. The service should abstain if applicable approved evidence is missing.

Is this production-ready after adding RAG? Not automatically. Persistence, concurrency, security, data handling, operational reliability and domain validation must each have evidence. A polished UI is not proof of readiness.

12. References and verification notes

The implementation choices above are recommendations for this codebase. Provider capabilities were checked against primary documentation; versions, account availability and prices must be rechecked when implementation starts.

- pgvector official documentation: vector types, exact/approximate search, index constraints and filtering.
- PostgreSQL 16 full-text search: lexical retrieval concepts.
- Gemini embeddings: embedding methods, models and task configuration.
- Gemini structured outputs: schema-constrained generation.
- Google Maps security guidance: application/API restrictions and credential separation.
- Google Compute Routes: road route request/response integration.

The source inspection did not run application tests, execute migrations, provision databases, call paid model/map APIs, ingest documents or alter current application code. The plans and estimates are proposed future work. The final Markdown and PDF are the only project deliverables from this task.